

yichuan-w / LEANN

<> Code

Issues 25

Pull requests 2

Discussions

Actions

Projects

Security

Insights

RAG on Everything with LEANN. Enjoy 97% storage savings while running a fast, accurate, and 100% private RAG application on your personal device.

MIT license

Contributing

2.7k stars

253 forks

24 watching

Branches

Activity

Tags

Public repository

main

46 Branches

27 Tags

Go to file

Go to file

Add file

<> Code

andyfizf

Introducing dynamic index update (#108) 5f7806e · 6 hours ago

.github	feat: Add GitHub PR and issue templates (#105)	3 days ago
.vscode	Auto-register apps: Universal index discovery (...)	last month
apps	[Fix] Enable AST chunking when installed (packag...	4 days ago
assets	[chore] add slack to share use case	3 weeks ago
benchmarks	[EXP] Update the benchmark code (#71)	last month
data	fix missing file	last month
docs	Ast fork (#92)	2 weeks ago
examples	Introducing dynamic index update (#108)	6 hours ago
packages	Introducing dynamic index update (#108)	6 hours ago
scripts	fix ruff errors and formatting	2 months ago
sky	Docs/Core: Low-Resource Setups, SkyPilot Optio...	last month
tests	Metadata filtering feature (#75)	last month
videos	add gif	2 months ago
.gitignore	docs(mcp): add root llms.txt for MCP discovery; u...	2 weeks ago
.gitmodules	change the submodule for easy pull	2 days ago
.pre-commit-config.yaml	[EXP] Update the benchmark code (#71)	last month
.python-version	Initial commit	3 months ago
LICENSE	upd readme	2 months ago
README.md	Add grep search functionality - Issue #86 (#87)	2 weeks ago
demo.ipynb	fix emaple and add pypi example	2 months ago
llms.txt	docs(mcp): add root llms.txt for MCP discovery; u...	2 weeks ago
pyproject.toml	Ast fork (#92)	2 weeks ago
uv.lock	Introducing dynamic index update (#108)	6 hours ago



Python 3.9 | 3.10 | 3.11 | 3.12 | 3.13 CI passing Platform Ubuntu & Arch & WSL | macOS (ARM64/Intel) License MIT MCP Native Integration Slack Join WeChat Join

The smallest vector index in the world. RAG Everything with LEANN!

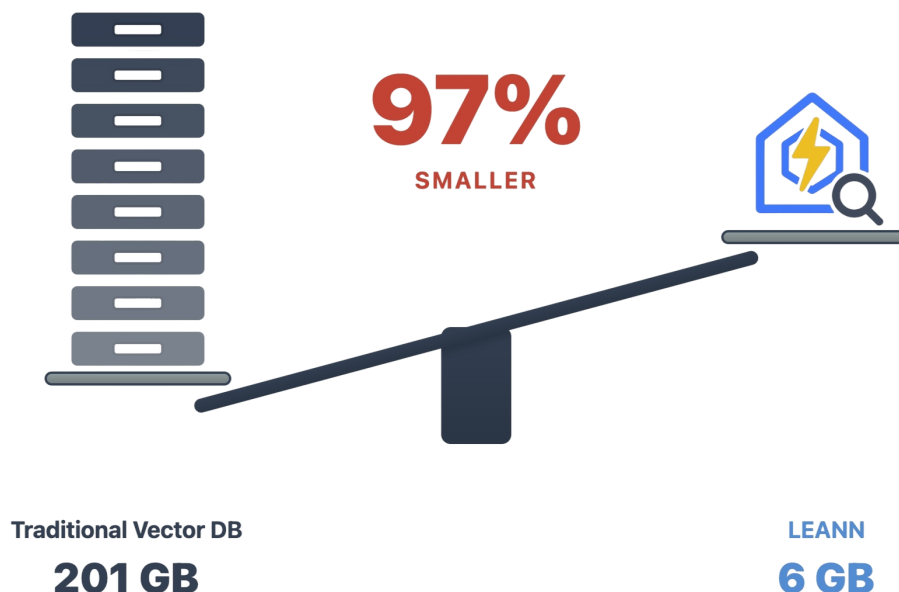
LEANN is an innovative vector database that democratizes personal AI. Transform your laptop into a powerful RAG system that can index and search through millions of documents while using **97% less storage** than traditional solutions **without accuracy loss**.

LEANN achieves this through *graph-based selective recomputation* with *high-degree preserving pruning*, computing embeddings on-demand instead of storing them all. [Illustration Fig →](#) | [Paper →](#)

Ready to RAG Everything? Transform your laptop into a personal AI assistant that can semantic search your [file system](#), [emails](#), [browser history](#), [chat history](#), [codebase](#)^{*}, or external knowledge bases (i.e., 60M documents) - all on your laptop, with zero cloud costs and complete privacy.

^{*} Claude Code only supports basic `grep`-style keyword search. LEANN is a drop-in **semantic search MCP service** fully compatible with **Claude Code**, unlocking intelligent retrieval without changing your workflow. 🔥 Check out [the easy setup →](#)

Why LEANN?



The numbers speak for themselves: Index 60 million text chunks in just 6GB instead of 201GB. From emails to browser history, everything fits on your laptop. [See detailed benchmarks for different applications below ↓](#)

🔒 **Privacy:** Your data never leaves your laptop. No OpenAI, no cloud, no "terms of service".

Lightweight: Graph-based recomputation eliminates heavy embedding storage, while smart graph pruning and CSR format minimize graph storage overhead. Always less storage, less memory usage!

📁 **Portable:** Transfer your entire knowledge base between devices (even with others) with minimal cost - your personal AI memory travels with you.

🚀 **Scalability:** Handle messy personal data that would crash traditional vector DBs, easily managing your growing personalized data and agent generated memory!

🌟 **No Accuracy Loss:** Maintain the same search quality as heavyweight solutions while using 97% less storage.

Installation

📦 Prerequisites: Install uv

[Install uv](#) first if you don't have it. Typically, you can install it with:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```



🔧 Quick Install

Clone the repository to access all examples and try amazing applications,

```
git clone https://github.com/yichuan-w/LEANN.git leann
cd leann
```



and install LEANN from [PyPI](#) to run them immediately:

```
uv venv
source .venv/bin/activate
uv pip install leann
```



▶ 🔧 Build from Source (Recommended for development)

Quick Start

Our declarative API makes RAG as easy as writing a config file.

Check out [demo.ipynb](#) or  [Open in Colab](#)

```
from leann import LeannBuilder, LeannSearcher, LeannChat
from pathlib import Path
INDEX_PATH = str(Path(".").resolve()) / "demo.leann"

# Build an index
builder = LeannBuilder(backend_name="hns")
builder.add_text("LEANN saves 97% storage compared to traditional vector databases.")
builder.add_text("Tung Tung Tung Sahur called—they need their banana-crocodile hybrid back")
builder.build_index(INDEX_PATH)

# Search
searcher = LeannSearcher(INDEX_PATH)
results = searcher.search("fantastical AI-generated creatures", top_k=1)

# Chat with your data
chat = LeannChat(INDEX_PATH, llm_config={"type": "hf", "model": "Qwen/Qwen3-0.6B"})
response = chat.ask("How much storage does LEANN save?", top_k=1)
```



RAG on Everything!

LEANN supports RAG on various data sources including documents (`.pdf` , `.txt` , `.md`), Apple Mail, Google Search History, WeChat, and more.

Generation Model Setup

LEANN supports multiple LLM providers for text generation (OpenAI API, HuggingFace, Ollama).

▶ 🔑 OpenAI API Setup (Default)

▶  Ollama Setup (Recommended for full privacy)

★ Flexible Configuration

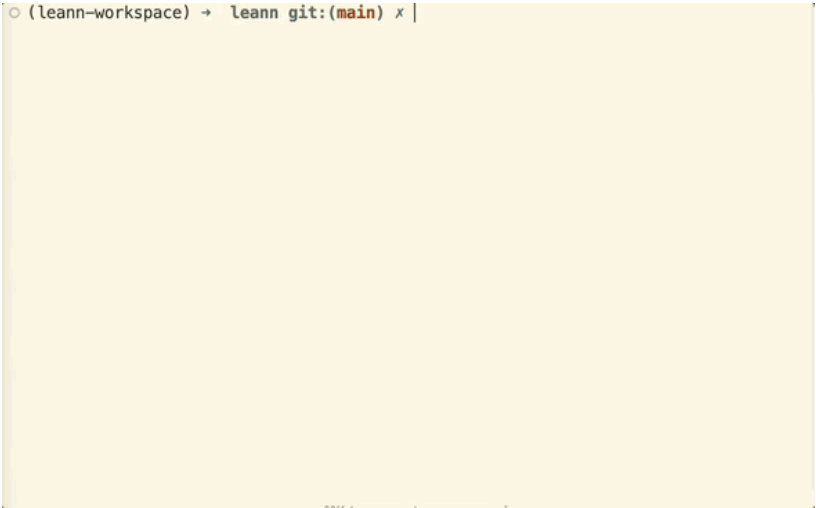
LEANN provides flexible parameters for embedding models, search strategies, and data processing to fit your specific needs.

 **Need configuration best practices?** Check our [Configuration Guide](#) for detailed optimization tips, model selection advice, and solutions to common issues like slow embeddings or poor search quality.

▶  Click to expand: Common Parameters (Available in All Examples)

Personal Data Manager: Process Any Documents (`.pdf` , `.txt` , `.md`)!

Ask questions directly about your personal PDFs, documents, and any directory containing your files!



The example below asks a question about summarizing our paper (uses default data in `data/` , which is a directory with diverse data sources: two papers, *Pride and Prejudice*, and a Technical report about LLM in Huawei in Chinese), and this is the **easiest example** to run here:

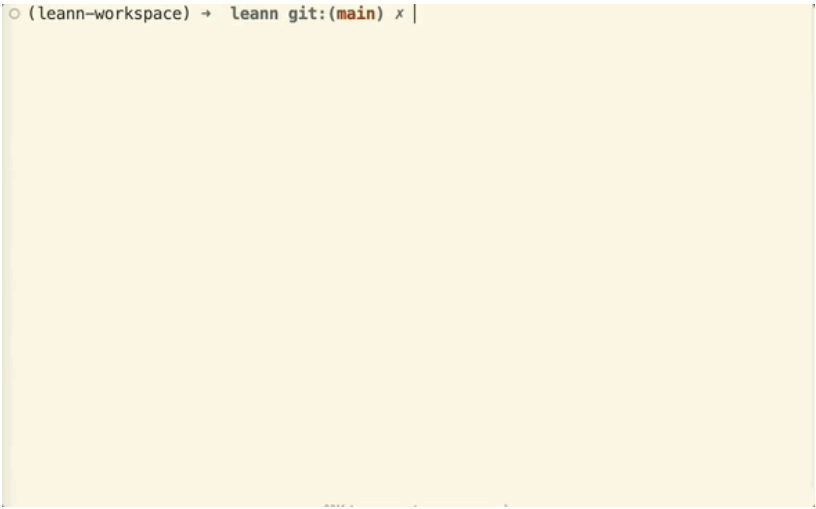
```
source .venv/bin/activate # Don't forget to activate the virtual environment
python -m apps.document_rag --query "What are the main techniques LEANN explores?"
```



▶  Click to expand: Document-Specific Arguments

Your Personal Email Secretary: RAG on Apple Mail!

Note: The examples below currently support macOS only. Windows support coming soon.



Before running the example below, you need to grant full disk access to your terminal/VS Code in System Preferences → Privacy & Security → Full Disk Access.

```
python -m apps.email_rag --query "What's the food I ordered by DoorDash or Uber Eats mostly?"
```



780K email chunks → 78MB storage. Finally, search your email like you search Google.

▶ Click to expand: Email-Specific Arguments

▶ Click to expand: Example queries you can try

Time Machine for the Web: RAG Your Entire Chrome Browser History!

```
(leann-workspace) → leann git:(main) x |
```

```
python -m apps.browser_rag --query "Tell me my browser history about machine learning?"
```



38K browser entries → 6MB storage. Your browser history becomes your personal search engine.

▶ Click to expand: Browser-Specific Arguments

▶ Click to expand: How to find your Chrome profile

▶ Click to expand: Example queries you can try

WeChat Detective: Unlock Your Golden Memories!

```
(leann-workspace) → leann git:(main) x python examples/wechat_history_reader_leann.py --index wechat_index --max 50 --query "whats the magic johnson's jersey I come across in wechat group"
```

```
python -m apps.wechat_rag --query "Show me all group chats about weekend plans"
```



400K messages → 64MB storage Search years of chat history in any language.

- ▶  Click to expand: Installation Requirements
- ▶  Click to expand: WeChat-Specific Arguments
- ▶  Click to expand: Example queries you can try

Claude Code Integration: Transform Your Development Workflow!

▶ NEW!! AST-Aware Code Chunking

The future of code assistance is here. Transform your development workflow with LEANN's native MCP integration for Claude Code. Index your entire codebase and get intelligent code assistance directly in your IDE.

Key features:

- 🔍 Semantic code search across your entire project, fully local index and lightweight
- AST-aware chunking preserves code structure (functions, classes)
- 🛠️ Context-aware assistance for debugging and development
- ⚙️ Zero-config setup with automatic language detection

```
# Install LEANN globally for MCP integration
uv tool install leann-core --with leann
claude mcp add --scope user leann-server -- leann_mcp
# Setup is automatic - just start using Claude Code!
```



Try our fully agentic pipeline with auto query rewriting, semantic search planning, and more:

```
• leann-server - leann_search (MCP)(index_name: "my-project", query: "cache reorder segment", top_k: 10)
  | INFO: Registering backend 'diskann'
  | INFO: Registering backend 'hns'
  | INFO: Registering backend 'diskann'
  | ... +165 lines (ctrl+r to expand)

Now let me search for the main research papers or documents in the project:

• Search(pattern: "*.pdf")
  | Found 0 files

• Search(pattern: "*.tex")
  | Found 0 files

• Search(pattern: "*.md")
  | Found 0 files

Let me search for more specific project files and look at the main directory structure:

• List(.)
  | Listed 8 paths (ctrl+r to expand)

Let me examine the PowerPoint presentations to understand the cache reorder and segment techniques:

• leann-server - leann_ask (MCP)(index_name: "my-project", question: "What are the main cache reorder and segment techniques used in DiskGNN? Describe the key
  | approaches for cache reordering and segmentation.")
  | INFO: Registering backend 'diskann'
  | INFO: Registering backend 'hns'
  | Starting chat with index 'my-project'...
  | ... +92 lines (ctrl+r to expand)

Based on the search results from the LEANN index, the main techniques about cache reorder and segment in DiskGNN are:

Cache Reordering Technique: MinHash-Based Reordering
```

🔥 Ready to supercharge your coding? [Complete Setup Guide →](#)

Command Line Interface

LEANN includes a powerful CLI for document processing and search. Perfect for quick document indexing and interactive chat.

Installation

If you followed the Quick Start, `leann` is already installed in your virtual environment:

```
source .venv/bin/activate
leann --help
```



To make it globally available:

```
# Install the LEANN CLI globally using uv tool
uv tool install leann-core --with leann
```

```
# Now you can use leann from anywhere without activating venv
leann --help
```

Note: Global installation is required for Claude Code integration. The `leann_mcp` server depends on the globally available `leann` command.

Usage Examples

```
# build from a specific directory, and my_docs is the index name(Here you can also build from multiple dict or multiple files)
leann build my-docs --docs ./your_documents
```

```
# Search your documents
```

```
leann search my-docs "machine learning concepts"
```

```
# Interactive chat with your documents
```

```
leann ask my-docs --interactive
```

```
# List all your indexes
```

```
leann list
```

```
# Remove an index
```

```
leann remove my-docs
```

Key CLI features:

- Auto-detects document formats (PDF, TXT, MD, DOCX, PPTX + code files)
- **AST-aware chunking** for Python, Java, C#, TypeScript files
- Smart text chunking with overlap for all other content
- Multiple LLM providers (Ollama, OpenAI, HuggingFace)
- Organized index storage in `.leann/indexes/` (project-local)
- Support for advanced search parameters

►  Click to expand: Complete CLI Reference

Advanced Features

Metadata Filtering

LEANN supports a simple metadata filtering system to enable sophisticated use cases like document filtering by date/type, code search by file extension, and content management based on custom criteria.

```
# Add metadata during indexing
builder.add_text(
    "def authenticate_user(token): ...",
    metadata={"file_extension": ".py", "lines_of_code": 25}
)

# Search with filters
results = searcher.search(
    query="authentication function",
    metadata_filters={
        "file_extension": {"==": ".py"},
        "lines_of_code": {"<": 100}
    }
)
```

Supported operators: `==`, `!=`, `<`, `<=`, `>`, `>=`, `in`, `not_in`, `contains`, `starts_with`, `ends_with`, `is_true`, `is_false`

 [Complete Metadata filtering guide →](#)

Grep Search

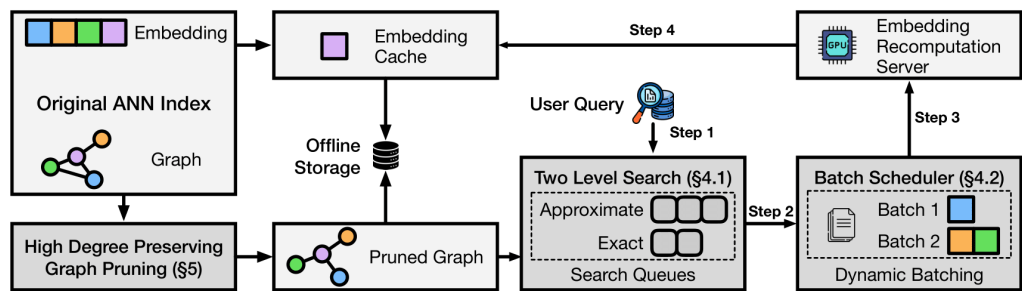
For exact text matching instead of semantic search, use the `use_grep` parameter:

```
# Exact text search
results = searcher.search("banana-crocodile", use_grep=True, top_k=1)
```

Use cases: Finding specific code patterns, error messages, function names, or exact phrases where semantic similarity isn't needed.

[Complete grep search guide →](#)

Architecture & How It Works



The magic: Most vector DBs store every single embedding (expensive). LEANN stores a pruned graph structure (cheap) and recomputes embeddings only when needed (fast).

Core techniques:

- **Graph-based selective recomputation:** Only compute embeddings for nodes in the search path
- **High-degree preserving pruning:** Keep important "hub" nodes while removing redundant connections
- **Dynamic batching:** Efficiently batch embedding computations for GPU utilization
- **Two-level search:** Smart graph traversal that prioritizes promising nodes

Backends:

- **HNSW** (default): Ideal for most datasets with maximum storage savings through full recomputation
- **DiskANN:** Advanced option with superior search performance, using PQ-based graph traversal with real-time reranking for the best speed-accuracy trade-off

Benchmarks

[DiskANN vs HNSW Performance Comparison →](#) - Compare search performance between both backends

[Simple Example: Compare LEANN vs FAISS →](#) - See storage savings in action

Storage Comparison

System	DPR (2.1M)	Wiki (60M)	Chat (400K)	Email (780K)	Browser (38K)
Traditional vector database (e.g., FAISS)	3.8 GB	201 GB	1.8 GB	2.4 GB	130 MB
LEANN	324 MB	6 GB	64 MB	79 MB	6.4 MB
Savings	91%	97%	97%	97%	95%

Reproduce Our Results

```
uv pip install -e ".[dev]" # Install dev dependencies
python benchmarks/run_evaluation.py # Will auto-download evaluation data and run benchmarks
```



```
python benchmarks/run_evaluation.py benchmarks/data/indices/rpj_wiki/rpj_wiki --num-queries 2000 # After downloading data, y
```

The evaluation script downloads data automatically on first run. The last three results were tested with partial personal data, and you can reproduce them with your own data!

Paper

If you find Leann useful, please cite:

[LEANN: A Low-Storage Vector Index](#)

```
@misc{wang2025leannlowstoragevectorindex,
  title={LEANN: A Low-Storage Vector Index},
  author={Yichuan Wang and Shu Liu and Zhifei Li and Yongji Wu and Ziming Mao and Yilong Zhao and Xiao Yan and Zhiying Xu and Yichuan Wang},
  year={2025},
  eprint={2506.08276},
  archivePrefix={arXiv},
  primaryClass={cs.DB},
  url={https://arxiv.org/abs/2506.08276},
}
```

✦✦ [Detailed Features →](#)

[CONTRIBUTING →](#)

? [FAQ →](#)

📈 [Roadmap →](#)

📄 [License](#)

MIT License - see [LICENSE](#) for details.

🧑‍🤝🧑 [Acknowledgments](#)

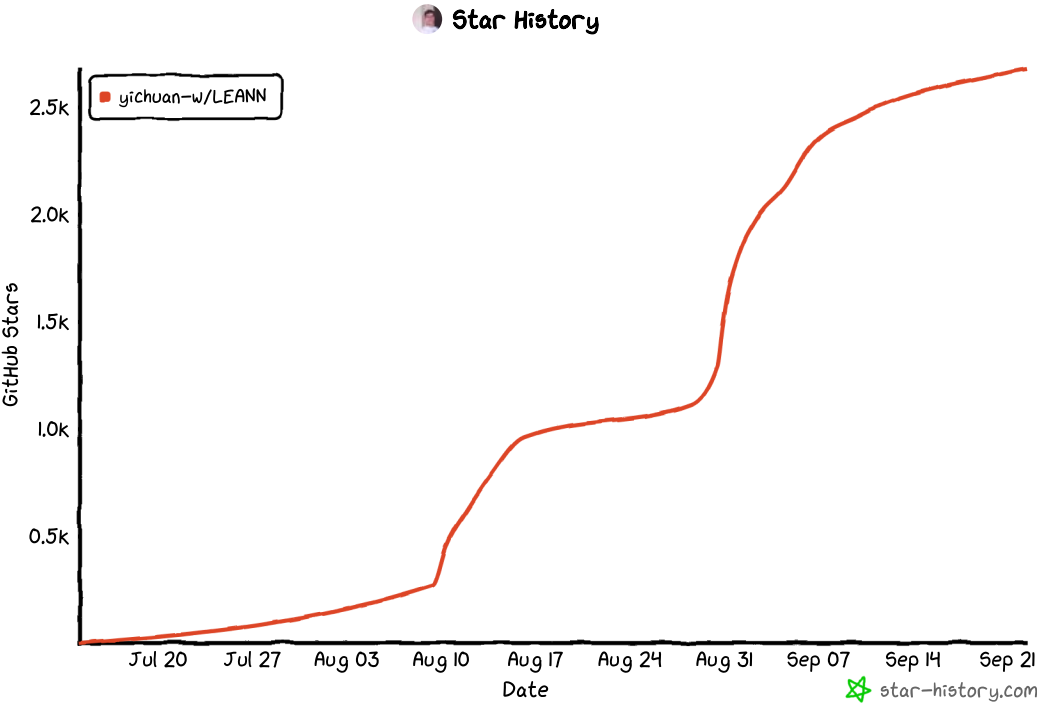
Core Contributors: [Yichuan Wang](#) & [Zhifei Li](#).

Active Contributors: [Gabriel Dehan](#)

We welcome more contributors! Feel free to open issues or submit PRs.

This work is done at [Berkeley Sky Computing Lab](#).

Star History



★ Star us on GitHub if Leann is useful for your research or applications!

Made with ❤ by the Leann team

Releases 26

Release v0.3.4

Latest

4 days ago

+ 25 releases

Packages

No packages published

Contributors 10



Languages

